A Model for Cloud Computing

- It is important to connect distributed computing models to practical models like the cloud.
- One step in that direction is the Massively Parallel Computing model, MPC model for short.
- The MPC model has a number of pairwise interconnected machines.
  - Similar to the All-to-All communication model
- However, we assume that no single machine can store the entire input by itself.
  - Space is not enough!
- We still measure the number of rounds of communication needed to solve a problem.

The MPC Model

- Let each machine have a space of S.
- Let N be the size of the input.
- We assume that N >> S.
- The input is distributed across the machines in equal proportion with each of the M = O(N/S) machines holding O(S) data.
- We assume that the number of machines M and the space per machine is in $O(N^{\delta})$ for some $\delta < 1$.

The MPC Model

- The model allows for synchronous communication rounds.
- In each round, each node can receive up to S words in total from all other nodes, perform local computation, and send up to a total of S words to other nodes.

- We measure the performance of an algorithm in the MPC model by the number of communication rounds needed by the algorithm.
- This measure ignores the time taken by local computation and seems unnatural.

A First Algorithm – Sorting

- Let us consider sorting N items in the MPC model.
- Let us consider having m machines each of space S.
- Assume that $S = N^{\delta}$, with $\delta > ½$.
- The input is spread across the machines with each machine having ni items.
- The algorithm we study resembles quick sort, except that we have more pivots.

Sorting

- let N_i refer to the set of input numbers stored at machine M_i .
- Let $n_i = |N_i|$
- Let each machine M_i choose each element as a pivot with probability p.
- Number of pivots = n_i . p.

Sorting in the MPC Model

- As the next step, let each machine send the set of pivots chosen to all other machines.
- Each machine then computes the local rank of each pivot.
- Rank(q, i) = the local rank of a pivot q in a machine M_i, = the number of elements in M_i smaller than the q.
- Each machine then sends the local ranks of all the pivots to a designated machine M.

Sorting

- Machine M uses these local ranks to identify a good set of $N \cdot p$ pivots.
- Machine M chooses one pivot for every N^{δ} elements of the input.
- Let the chosen pivots be $Q := \{q1, q2, \cdots, q_{N^{1-\delta}}\}$.
- Machine M then sends Q to all other machines.

Sorting

- Each machine now partitions its input into |Q| + 1 buckets, with bucket j having elements that have a value between the (j − 1)th and j th pivot (of Q).
- In a final round of communication, machines exchange elements such that all elements x currently at machine M_i with a value between q_{j-1} and q_j are sent to machine M_j . (Use q0 = −∞ and q|Q|+1 = ∞).
- At this point, machine Mj has a set of input numbers with a value between q_{j-1} and q_j .
- This machine can then sort these numbers locally and assign ranks to these numbers starting from the global rank of pivot q_{j-1} + 1 to the global rank of q_j.

Sorting

- Lemma: With high probability, some machine marks at least one element in every segment of $N^{1-\delta}$ ranks of the input A.
- The probability that no machine marks any of the elements in a given segment of $N^{1-\delta}$ ranks of the input array is $(1 - p)^{N^{1-\delta}}$.
- This probability is at most $e^{-pN^{1-\delta}}$.

Sorting

- Lemma: With high probability, some machine marks at least one element in every segment of $N^{1-\delta}$ ranks of the input A.
- This probability is at most $e^{-pN^{1-\delta}}$.
- Setting $p = 4S \log N / N$, the above probability is at most $e^{-4 \log N} = 1/N^4$.
- There are at most N segments of length $N^{1-\delta}$.
- We now use the union bound over all segments to conclude that with probability at least $1-1/N^3$, in every segment of $N^{1-\delta}$, there exists at least one element marked as a pivot.

Sorting

- An important component of this analysis is to check the communication volume since, in each round, each machine can send and receive up to O(S) words.
- Let us analyze the number of communication rounds the algorithm needs.

Sorting

- Each machine selects $O(S \cdot p)$ pivots.
- Thus, it sends and receives $O(S \cdot p)$ pivots from each of the other machines, on expectation.
- The total number of words that a machine receives is in $O(N /S \times S \cdot p) = O(N \cdot p)$.
- With $p = 4S \log N /N$, the communication volume at each machine is in $O(S)$.
- Further, the last communication step of rearranging elements also has a communication volume of $O(S)$ at each machine.

Sorting

Algorithm 1 Quicksort in MPC.

1: Each machine M_i marks each element of S_i as a pivot with probability p;
2: Each M_i sends all the pivots chosen by M_i to all other machines
3: Each machine M_i calculates the local rank of each pivot, $\mathrm{LocalRank}(q, S_i)$.
4: Each machine M_i sends its LocalRank to a designated machine M, say M_1
5: Machine M_1 uses the arrays to designate Np pivots and their global ranks, $\mathrm{Rank}(q, S)$ and communicates the pivots and Rank to all machines
6: Each machine M_i sends the elements with a value between q_{j-1} and q_j to machine M_j
7: Each machine M_i sorts S_i locally and assigns a rank

Sorting

- Overall Result: We need O(1) rounds with high probability to sort N elements in the MPC model.

- Other problems such as median can be solved in a similar fashion.

The MPC Model

- The MPC model makes more interesting study when the input is a large graph.
- When the input is a graph, notice that N = O(m+n).
  - We use n to denote the number of nodes in the graph
  - We use m to denote the number of edges in the graph.
- There are three variants studied in this context based on the relation between n and S.

The MPC Model

- The super-linear regime: Here, $S = \Omega(n^{1+\varepsilon})$.
- We still assume that $m >> S$.
- So, each machine has enough space to store information about all vertices and more than that.
- How is this additional space helpful?
- Let us study with the MST problem again.

The MPC Model

- The super-linear regime: Here, $S = \Omega(n^{1+\varepsilon})$.
- We still assume that $m >> S$.
- Let us study with the MST problem again.
- Partition the edges of the graph into $E_1, E_2, \ldots, E_r$ such that $|E_i| = n^{1+\varepsilon}$.
- Compute $T_i = MST(E_i)$.
- Recursively compute and return the MST of $U_i T_i$.

The MPC Model

- How many recursive calls are needed?
- Assume $m = n^{1+c}$.
- So, initially, there are $m/n^{1+\varepsilon} = n^{c-\varepsilon}$ sets of edges.
- After computing T_i with at most n edges each, the number of edges in $U_i T_i$ is at most $nxn^{c-\varepsilon} = n^{1+c-\varepsilon}$.
- So, one recursive call reduces the number of edges from n^{1+c} to $n^{1+c-\varepsilon}$.

The MPC Model

- How many recursive calls are needed?
- So, one recursive call reduces the number of edges from n^{1+c} to $n^{1+c-\varepsilon}$.
- In the next call, the number of edges in Ui Ti will go down from $n^{1+c-\varepsilon}$ to $n^{1+c-2\varepsilon}$.
- In c/ε such recursive calls, the number of edges falls to a quantity where a single machine can store all the edges.
- Algorithms that run in $O(1/\varepsilon)$ rounds are known for several other graph problems too in this model.

The Linear MPC Model

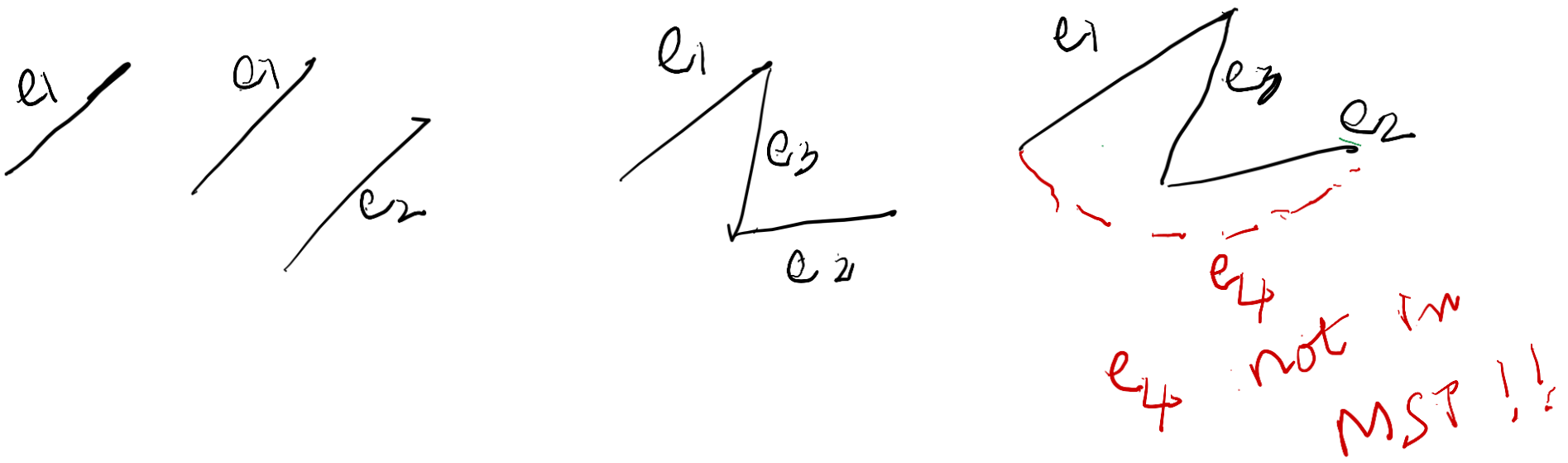
- Having $S = n^{1+\varepsilon}$ makes it very easy to design algorithms that are super-fast.
- Let us reduce S to $\Theta(n)$.
- We call this the linear MPC model.
- In the linear model, each machine has enough space to store a very limited information about each node in the graph.
- Let us continue the MST example and see how to design algorithms in this model.

The Linear MPC Model

- The techniques needed here are quite involved.
- Consider a sorted order of edges by weight. Let the ordering be $e_1, e_2, \ldots, e_m$.
- The following observation holds:

  Edge e_i is in the MST if and only if its endpoints are not in the same connected components in the graph with edge set $\{e_1, \ldots, e_{i-1}\}$.
- We can use this observation directly to convert the MST problem to the connected components problem.
  - Check the observation for each of e_1 to e_m.
- However, this algorithm is quite wasteful.

The Linear MPC Model



- The techniques needed here are quite involved.
- Consider a sorted order of edges by weight. Let the ordering be $e_1, e_2, \ldots, e_m$.
- The following observation holds:

  Edge e_i is in the MST if and only if its endpoints are not in the same connected components in the graph with edge set $\{e_1, \ldots, e_{i-1}\}$.

The Linear MPC Model

- One improvement is as follows.
- Group the m edges into m/n groups of n edges each.
- Call $E_i = \{e_{(i-1)n+1}, \ldots, e_{in}\}$ for i = 1 to m/n.
- Let $H_i = U_{j=1}^{i} E_j$. H_i is the union of the first i E_is for i=1 to m/n.
- Let F_i be the MST (Forest) corresponding to H_i.
- Notice that $|F_i| = O(n)$.
- So, given F_i and E_{i+1}, both of size O(n) edges, one machine can check which edges in E_{i+1} will be in the MST.
- We could compute each F_i in parallel!

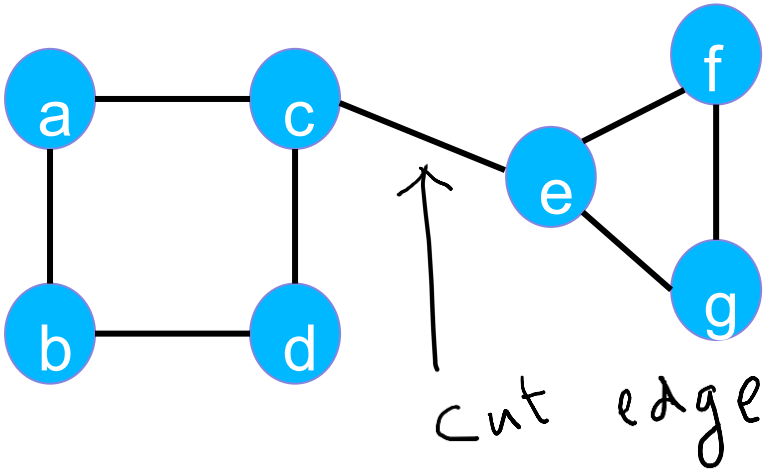
The Linear MPC Model

- Call $E_i = \{e_{(i-1)n+1}, \ldots, e_{in}\}$ for i = 1 to m/n.
- Let $H_i = U_{j=1}^{i} E_j$. H_i is the union of the first i E_is for i=1 to m/n.
- Let F_i be the MST (Forest) corresponding to H_i.
- Notice that $|F_i| = O(n)$.
- So, given F_i and E_{i+1}, both of size O(n) edges, one machine can check which edges in E_{i+1} will be in the MST.
- We could compute each F_i in parallel!
- But, $|H_i|$ is too big to fit in any single machine!!

The Linear MPC Model

- We just wish that computing the F_is takes as few rounds as possible.
- Here is a way to do that.
- We use the technique of graph sketching.
- We then explain how to implement the technique in the linear MPC model to compute connected components in O(1) rounds.

The Linear MPC Model



$A = \{a, b, c, d\}$

- Let A be a subset of V. Let C be the cut between A and V \ A.
- Assume for a moment that the cut has only edge crossing it.
- We wish to find the cut edge across A and V \ A using only O(log n) bits of memory.
- We use a bit trick that works as follows.

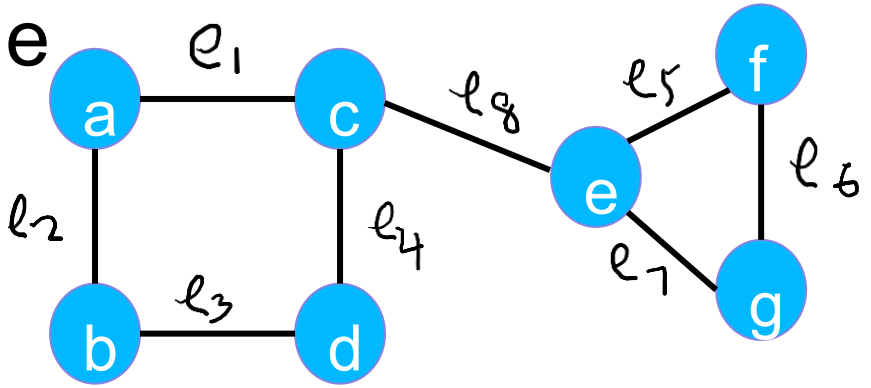
The Linear MPC Model

- Identify each edge by the concatenation the identifiers of its endpoints. For e = uv, id(e) = u o v if id(u) < id(v).
- Locally, each vertex u computes

  $XOR_u = \bigoplus_{e \in E, e \text{ has } u \text{ as an end point}} id(e)$.

The Linear MPC Model

- Example



Edge	End points	ID
e1	a, c	000010
e2	a, b	000001
e3	b, d	001011
e4	c,d	010011
e5	e, f	100101
e6	f, g	101110
e7	e, g	100110
e8	c, e	010100

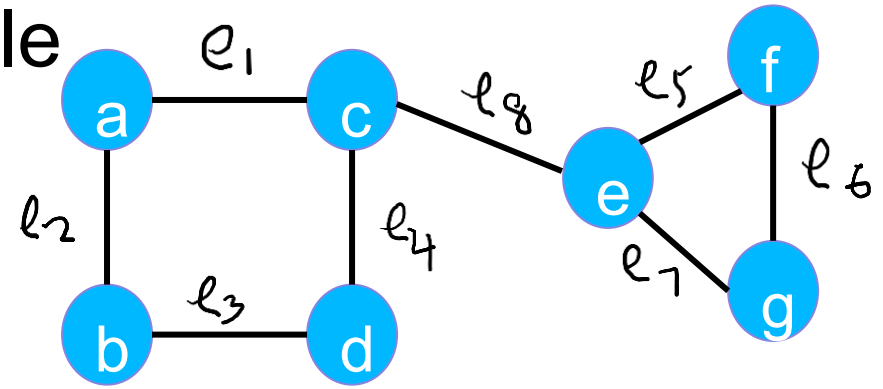
Node	Edges	XOR_u
a	e1, e2	000011
b	e2, e3	001010
c	e1, e4, e8	000101
d	e3, e4	011000
e	e5, e7, e8	100111
f	e5, e6	001011
g	e6, e7	111000

The Linear MPC Model

- Now, consider the XOR of all the vertices in A, $XOR_A = \bigoplus_{u \in A} XOR_u$.
- In XOR_A, an edge e = uv with both end points A appears in both XOR_u and XOR_v.
- On the other hand, the edge e = uv with u in A and v in V \ A, appears only once (in XOR_u).
- So, the result of XOR_A = id(e) thereby allowing us to identify the cut edge.

The Linear MPC Model

- Example

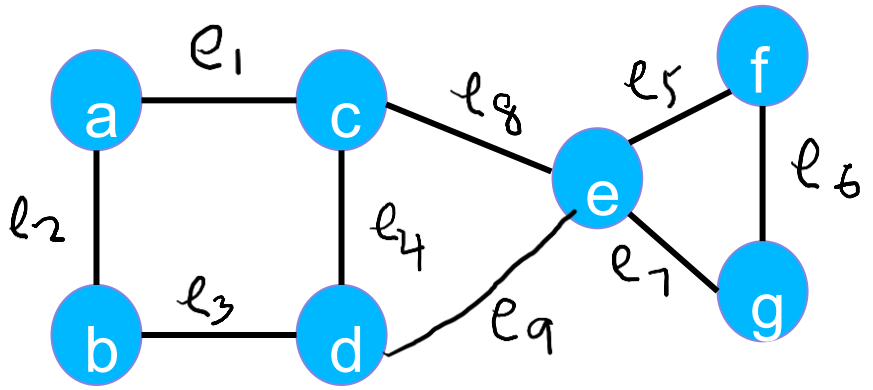


Edge	End points	ID
e1	a, c	000010
e2	a, b	000001
e3	b, d	001011
e4	c,d	010011
e5	e, f	100101
e6	f, g	101110
e7	e, g	100110
e8	c, e	010100

Node	Edges	XOR_u
a	e1, e2	000011
b	e2, e3	001010
c	e1, e4, e8	000101
d	e3, e4	011000
e	e5, e7, e8	100111
f	e5, e6	001011
g	e6, e7	111000

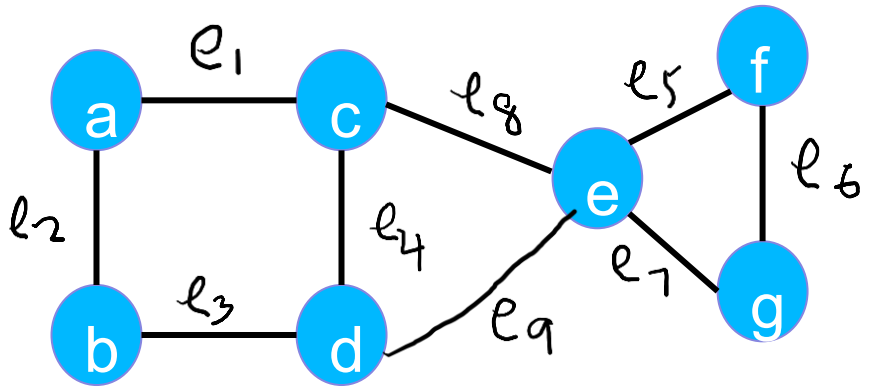
XOR_A = XOR(000011, 001010, 000101, 011000) = 010100 = ID(e8)

The Linear MPC Model



- We need to extend the above idea for the case where the cut has multiple cross edges.
- There are two problems to be solved.
- One is to see whether we are dealing with one cut edge or multiple cut edges.
- Second, is to extend the scheme to work for multiple cut edges.

The Linear MPC Model



- One is to see whether we are dealing with one cut edge or multiple cut edges.
- If the cut has more than one edge, then all such edges appear only once in XOR_A.
- The resulting XOR_A may actually end up being the id of some edge that is not a cut edge!
- So, we need a couple more techniques.

The Linear MPC Model

- Our first technique is to replace the id of vertices with O(log n) bit random strings of {0, 1}.
- XOR_A is now essentially a uniformly chosen bit string of the same length as the id of an edge.
- With this,

$$\Pr(\text{There exists an edge } e,\ XOR_A = id(e)) \leq \Sigma_e \Pr(XOR_A = id(e)) \leq {}^{n}C_2 \times (1/2)^{12\log n} = 1/n^{10}.$$

- This low probability helps us tide over the trouble of using actual vertex identifiers.

The Linear MPC Model

- For our second problem, we do the following.
- We do not know the number of edges crossing the cut.
- But, the number lies between 0 to m.
- So, we will try all possible estimates for the above number. All in parallel!
- The estimate is good if the actual number of edges crossing the cut is within ½ to 2 of the estimated number.
- In essence, we try estimates such as {1, 2, 4,…, m}.

The Linear MPC Model

- Let k be the actual number of cut edges.
- Run each estimate k' in [1, 2, 4, …, m] in parallel.
- With a given k', we mark edges with a probability of 1/k'.
- Consider the k' such that k'/2 <= k <= k'
- With a marking probability of 1/k', the expected number of cut edges marked is 1.
- We can reuse the case of identifying one cut edge if we get to mark just one of the k cut edges and no other edges are marked.
- Let us calculate the probability for the above event.

The Linear MPC Model

- Let S be the set of cut edges marked.

$$\Pr(|S| = 1) = k \,.\, (1/k') (1-1/k')^{(k-1)}$$
$$\geq (k'/2) \,.\, (1/k') (1-1/k')^{(k-1)}.$$
$$\geq (1/2) \,.\, (1/4) \text{ as } 1 - x >= 4^{-x}.$$
$$\geq 1/8.$$

The Linear MPC Model

- A few more nitty-gritty details are needed beyond the two techniques.
- In particular, we will need multiple independent runs with each k'.
- With a success probability of at least 1/8, Chernoff bounds tell us that O(log n) runs will help us.
- The overall technique is called graph sketching.

The Linear MPC Model

- Since the memory needed per sketch per node is $O(\log^3 n)$, the linear MPC model can support the technique.
- Read more details from the posted resources.

- Putting together everything, we get:
- The MST of a graph G can be obtained in O(1) rounds in the linear memory MPC model.

The Sub-Linear Model

- We study one more regime, the sublinear MPC model.
- In the sub-linear model, each machine has space in $O(n^{\varepsilon})$ for a small $\varepsilon < 1$.
- So, in a machine, one cannot even store O(1) information per every node of the graph.
- Let us continue the MST example and see how to design algorithms in this model.

The Sub-linear MPC Model

- In the sub-linear MPC model, the only best known solution for the MST problem so far is to use the Boruvka algorithm.
- Recall that Boruvka's algorithm is what is used in the GHS algorithm also.
- The round complexity in that case is O(log n).

Boruvka in sub-linear Model

- We will outline how to implement the algorithm of Boruvka in O(log n) rounds in the MPC model.
- The solution uses convergecast and broadcast.
- However, care is needed to make sure that the communication volume does not exceed $O(n^{\varepsilon})$ in any round.

Boruvka – GHS – Minimum Spanning Tree

- Recall that the algorithm operates in phases.
- In each phase, the algorithm has two operations
  - Find MOE of all fragments, and
  - Merging fragments via their MOEs.
- The GHS algorithm described above correctly a distributed MST in O(n log n) rounds and uses O(m + n log n) messages.

The MPC Implementation

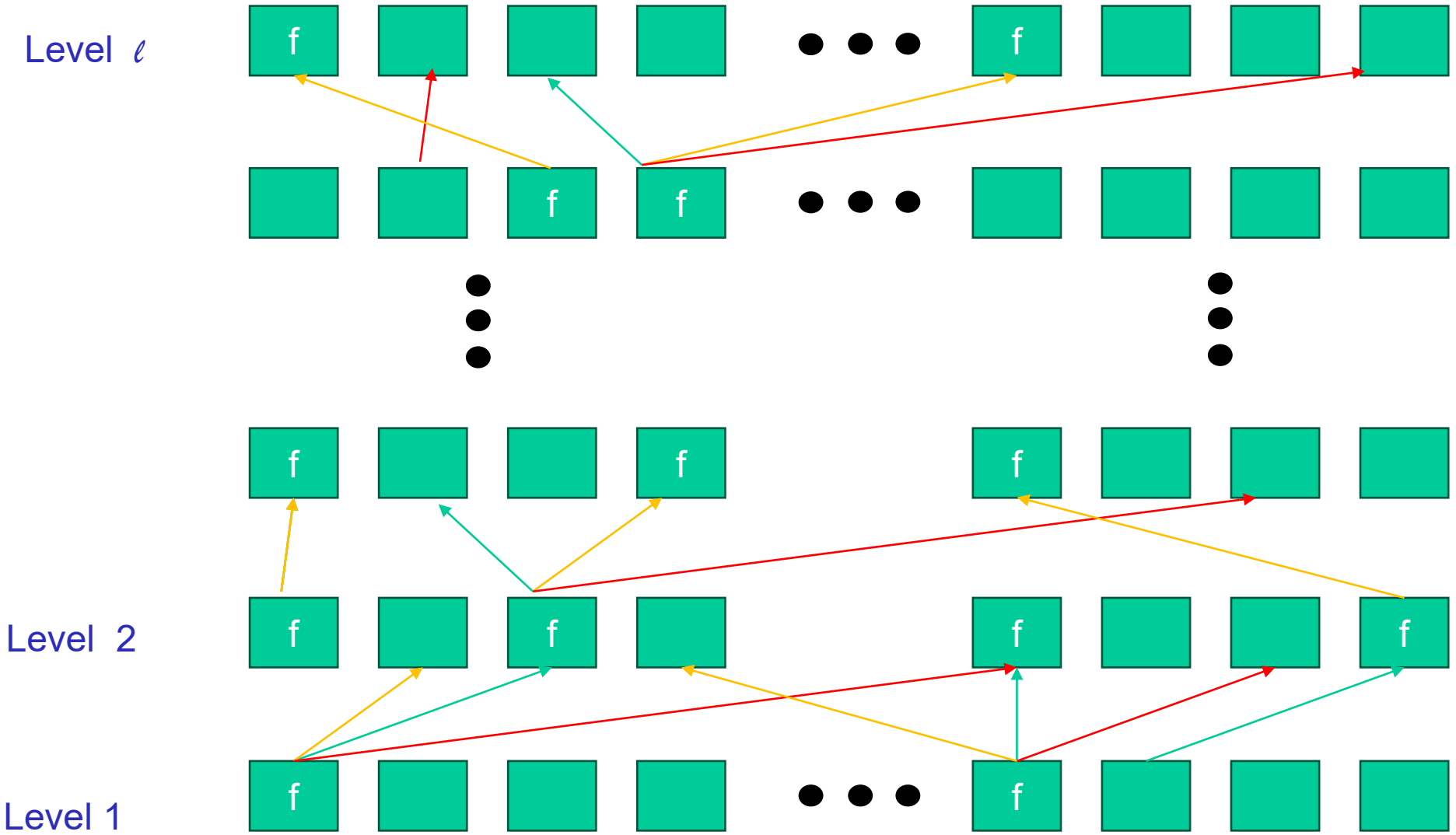
- The real challenge is to see how each phase of the algorithm can be implemented in O(1) rounds of the low-memory MPC model.
- All-to-all communication does not help since the volume of communication can be more than $O(n^{\varepsilon})$.
- Another issue could be a long chain of fragments that merge into one another.
- We solve these two problems using randomization.

MPC Implementation

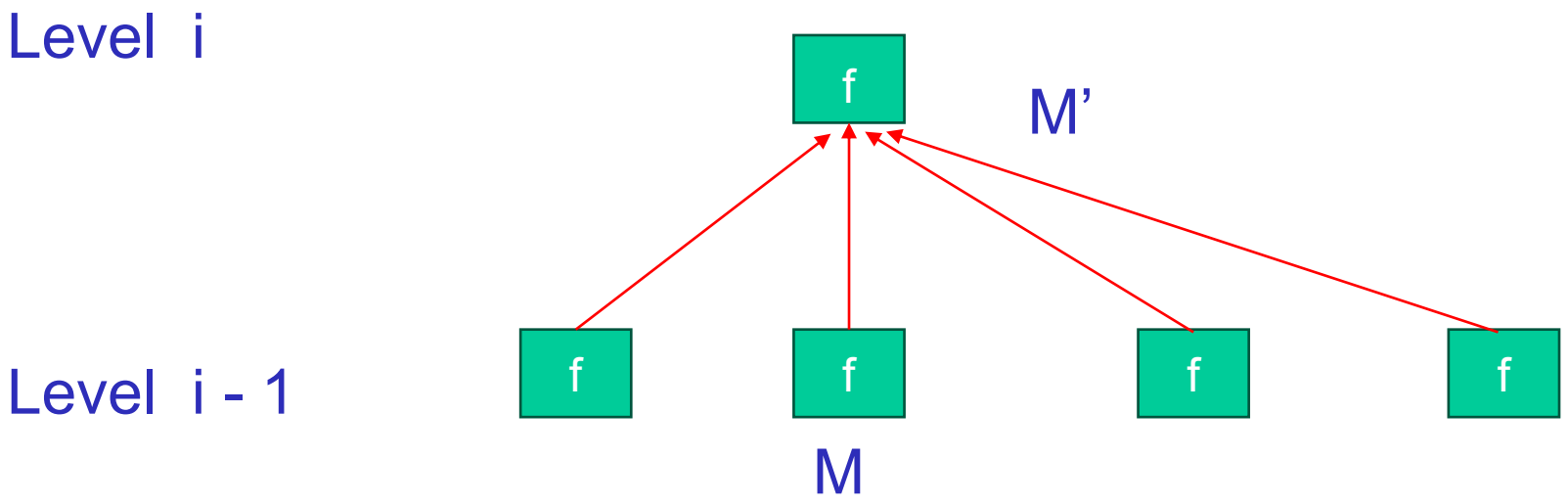
- Let S refer to the space per machine.
- Arrange the set of machines such that at level 1, we have all the m/S machines, each storing S edges.
- We say that a machine M works for an MSF fragment F if M stores an edge in F.
- At level 1, machine M works for a fragment F if M has an edge uv that belongs to fragment F.
- For a level i > 1, we say that a machine M on a level i works for a fragment F if M participates in the computation for the fragment F.

MPC Implementation

- Across two consecutive levels i and i+ 1, with i ≥ 1, define a parent-child relationship across machines as follows.
- Let ℓ be the largest level number
- For each fragment F and each level i for $1 \leq i \leq \ell$, each machine M on level i that works for fragment F has a parent machine M′ with respect to fragment ID F on level i + 1 if M′ also works for fragment F.
- On level i ≥ 2, a machine M is responsible for fragment F if a machine M exists on level i − 1 such that M is the parent with respect to fragment F of level i − 1.


Level ℓ
Level 2
Level 1
f

MPC Implementation

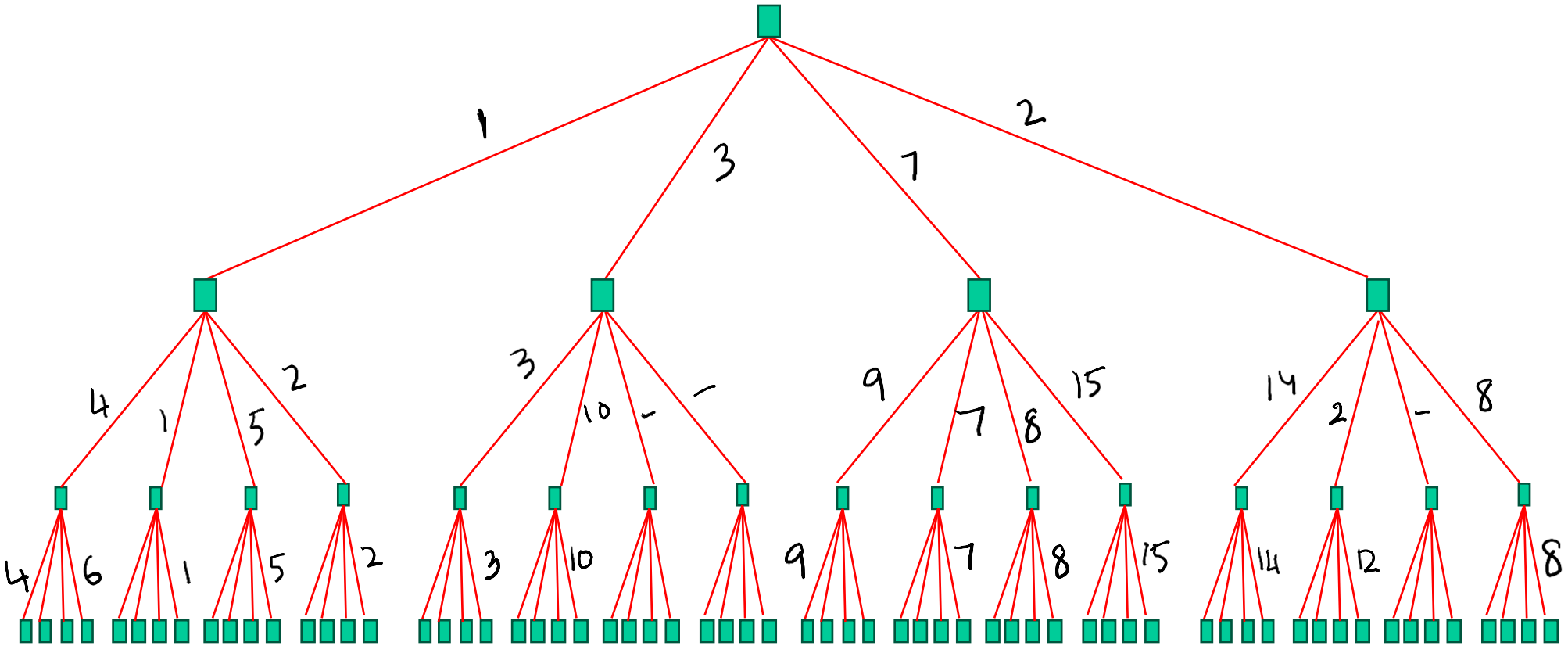


- The in-degree of a machine M′ on level i > 1 is the number of pairs (M, F) such that M′ is the parent of machine M with respect to fragment F on level i − 1.

MPC Implementation

- For each fragment F, each machine M on level 1 sends the minimum weight edge with exactly one endpoint in F that it stores to its parent.
- Continuing this level-by-level traversal, each machine on a level $i \geq 2$ waits to receive information from its children i level $i-1$.
- They can then send the required information to their parent with respect to each fragment F.
- This is akin to the convergecast operation.

MPC Implementation



MPC Implementation

- Further, let uv be the minimum weight each that the machine on the top level receives for a fragment F, with u in F and v in another fragment F′ .
- At the last level, the machine in level ℓ adds edge uv to the MST and initiates the change of representative from fragment F to that of fragment F′.
- This machine then broadcasts this change in representative to its children with respect to fragment F.
- This is akin to using the broadcast operation in the GHS algorithm.

Analysis

- Claim 1: The number of levels, ℓ, is O(1).
- Proof: The structure has a fan out of m/4, the number of levels is in $O(\log_{S/4} m) = \log m/\log(S/4)$.
- Since $S \in n^{\delta}$, and $m < n^2$, the ratio is in $O(8/\delta)$ provided $S > 16$.

- Claim 1 implies that the upward and downward traversals along the structure require O(1) communication rounds.

Analysis

- Claim: The in-degree of any machine at any given level is S/2.
- Proof: To this end, consider a machine M in a level $i \geq 1$, and a fragment F.
- Note that $m/(S/4)^i$ machines are in level i.
- So, the probability that machine M has to work for fragment F is $m/(S/4)^i$ / $m/(S/4))$ which is $(4/S)^{i-1}$.
- Given that M is working for fragment F, we bound the expected number of children for M as the ratio of the number of machines in level $i - 1$ working for fragment F over the number of machines in level i working for fragment F.
- This ratio is S/4, which is the fanout of the structure.

Analysis

- Continuing the proof of Claim 2, the unconditional expected number of children of M with respect to fragment F is thus $(4/S)^{i-1} \times (S/4) = (4/S)^{i-2}$.

- We now bound the expected in-degree of machine M as the product of the number of fragments with the expected number of children with respect to a given fragment.
- Thus, the expected indegree of machine M is at most $n \times (4/S)^{i-2} = S/2$, as $S = n^{\varepsilon}$.
- If $e > ½$, we can simplify the above to $(1/S)^{1/\varepsilon - i + 2}$.
- Claim 2 helps also bound the communication volume at any machine to be within S.

Analysis

- We now bound the expected in-degree of machine M as the product of the number of fragments with the expected number of children with respect to a given fragment.
- Thus, the expected indegree of machine M is at most $n \times (4/S)^{i-2} = S/2$, as $S = n^{\varepsilon}$.
- We can simplify the above to $(1/S)^{1/\varepsilon - i + 1}$.
- With i at most ℓ, and ℓ at most 4/e, the quantity can be seen to be at most O(S).
- Claim 2 helps also bound the communication volume at any machine to be within S.

Analysis

- Using Claims 1 and 2, we infer that each phase of the algorithm can run in O(1) rounds.
- With O(log n) phases, the overall rounds needed is O(log n).

Summary of the MPC model and MST

- Super-linear memory: $O(1/\varepsilon)$ rounds, easy technique.
- Linear memory: O(1) rounds, randomized, but very complicated and lots of techniques.
- Sub-linear memory: O(log n) rounds, with a complicated simulation and analysis.